Sistemas Operacionais

Sistema Operacional como Máquina Estendida e Gerenciador de Recursos

Rafael Descio, M.Sc.

Católica SC
Centro Universitário

Objetivos da Aula de Hoje

- **Entender** o SO como máquina estendida
- **Compreender** o SO como gerenciador de recursos
- **Identificar** abstrações: processos, arquivos e dispositivos
- **Reconhecer** chamadas de sistema em exemplos de código

Para pensar:

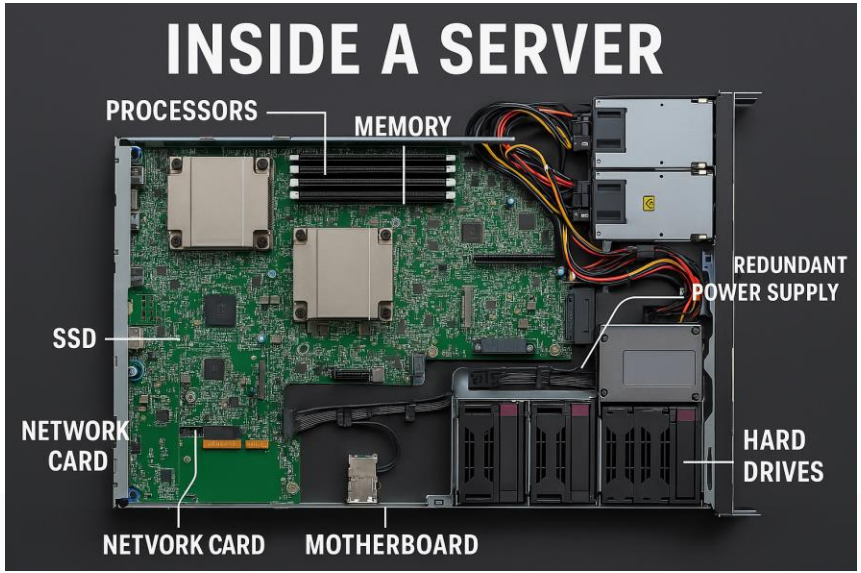
Como seria programar um computador sem um sistema operacional?

- Manipular disco diretamente
- Controlar memória manualmente
- Gerenciar dispositivos de hardware

Sistema Operacional como Máquina Estendida

O sistema operacional esconde a complexidade do hardware e fornece abstrações mais simples ao programador.

- Circuitos eletrônicos complexos
- Múltiplos dispositivos
- Comunicação entre componentes
- Controle de memória e CPU



O Problema do Hardware

Computadores possuem hardware complexo:

- Registradores
- Controladores de disco
- Interrupções
- Memória física
- Barramentos

Programar diretamente nesse nível seria **extremamente difícil**.

Exemplo: Registradores

Para que a CPU trabalhe de forma eficiente, tanto para interpretar um código de máquina, como para tratar a memória, ela usa os que chamamos de **Registradores**. Exemplo 8086, 14 registradores:

Dados:

AX - Acumulador

BX - Base

CX - Contador

DX – Dados

Segmentos:

CS - Segmento de Código

DS - Segmento de dados

SS - Segmento de Pilha

ES - Segmento Extra

Ponteiros e Índices:

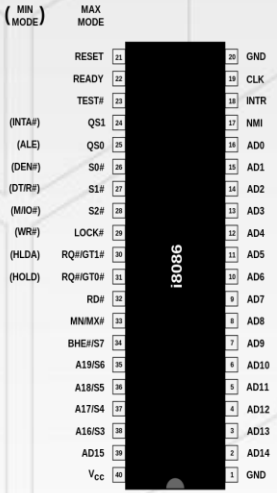
IP - Contador de Programa

SP - Ponteiro de Pilha

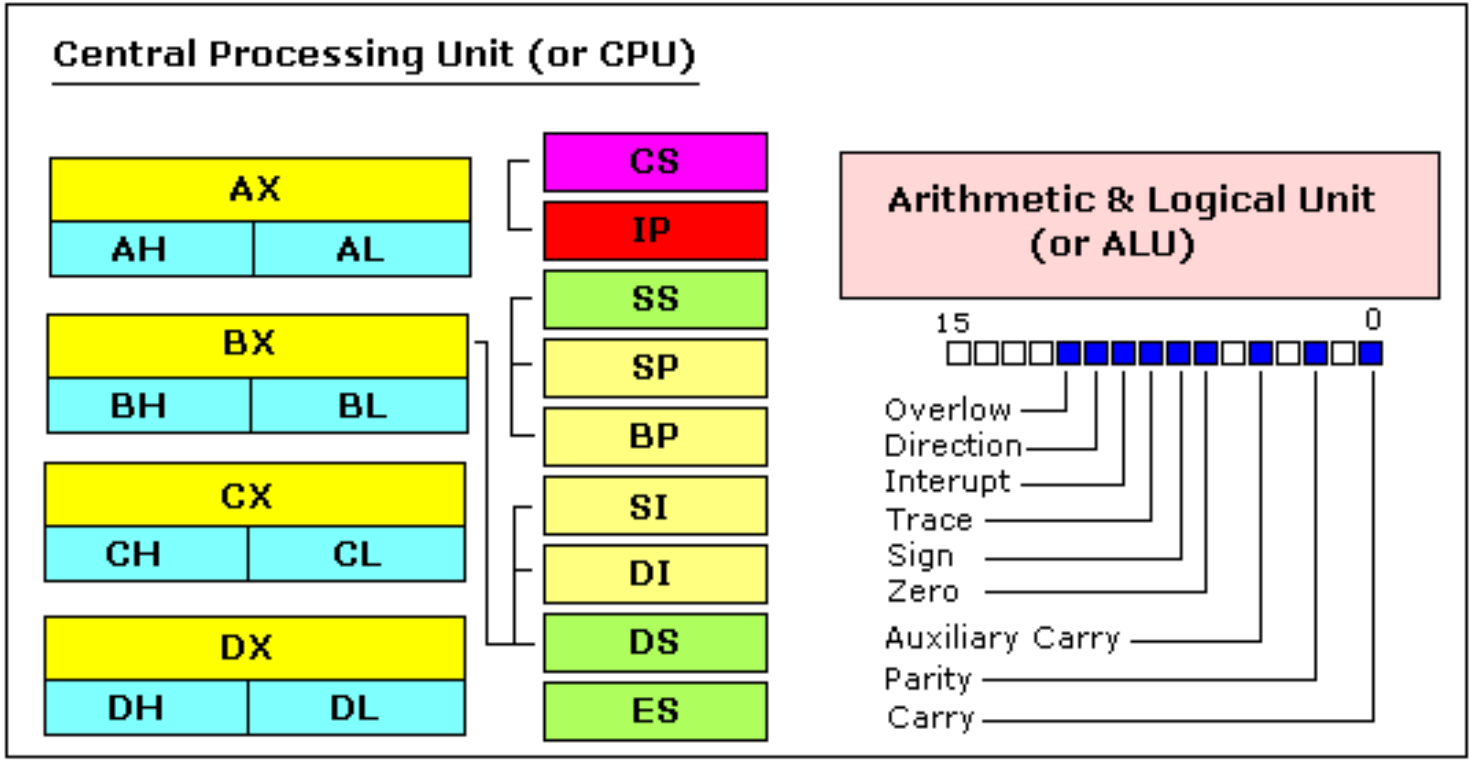
BP - Base de Pilha

SI - Indexador de Origem

DI - Indexador de Destino



Exemplo: Registradores



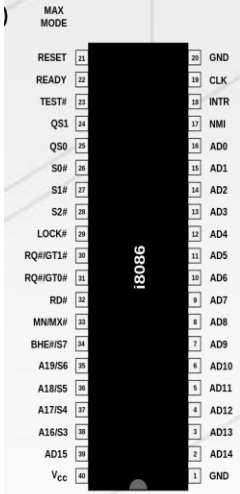
Exemplo: Registradores

The 16 bits of the flag register:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	R	R	R	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF

R = reserved
U = undefined
OF = overflow flag
DF = direction flag
IF = interrupt flag
TF = trap flag

SF = sign flag
ZF = zero flag
AF = auxiliary carry flag
PF = parity flag
CF = carry flag

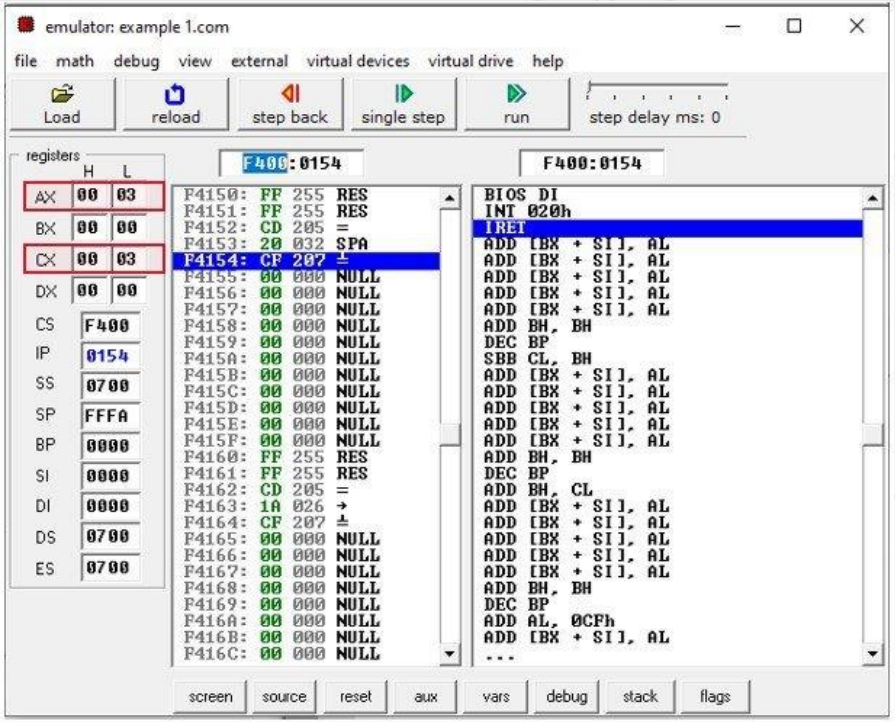


Exemplo: Código Assembly

Exemplo de código asm:
</> assembly

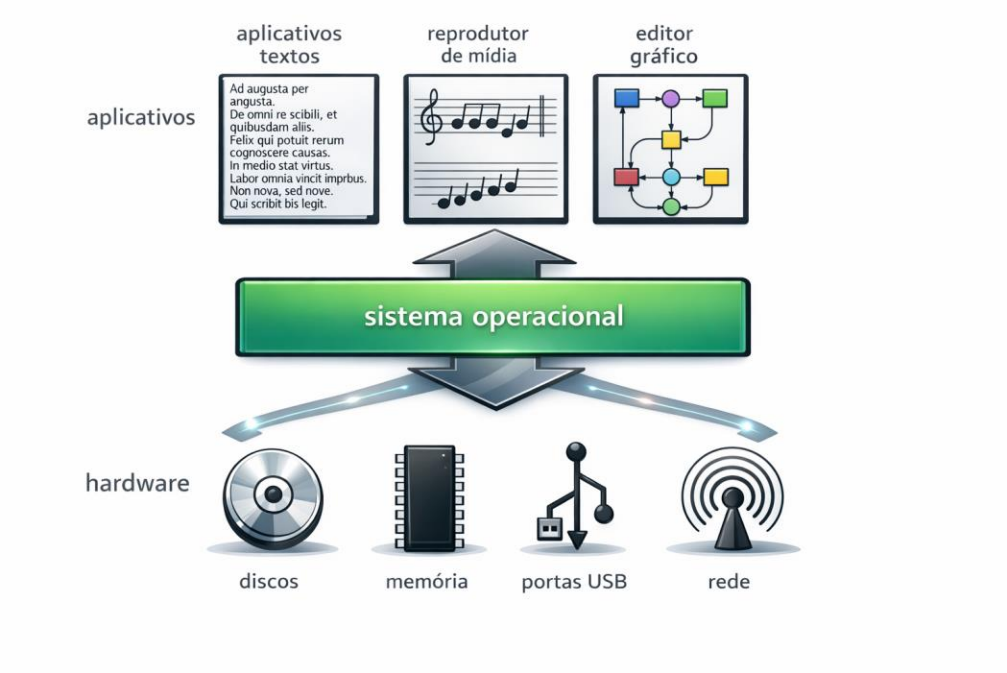
MOV AX, 5
MOV BX, 10
ADD AX, BX

Resultado:
AX = 15



O Papel do Sistema Operacional

O Sistema Operacional atua como uma **camada intermediária**. Realizando a “tradução” entre o programa e o hardware.



O Conceito de Máquina Estendida

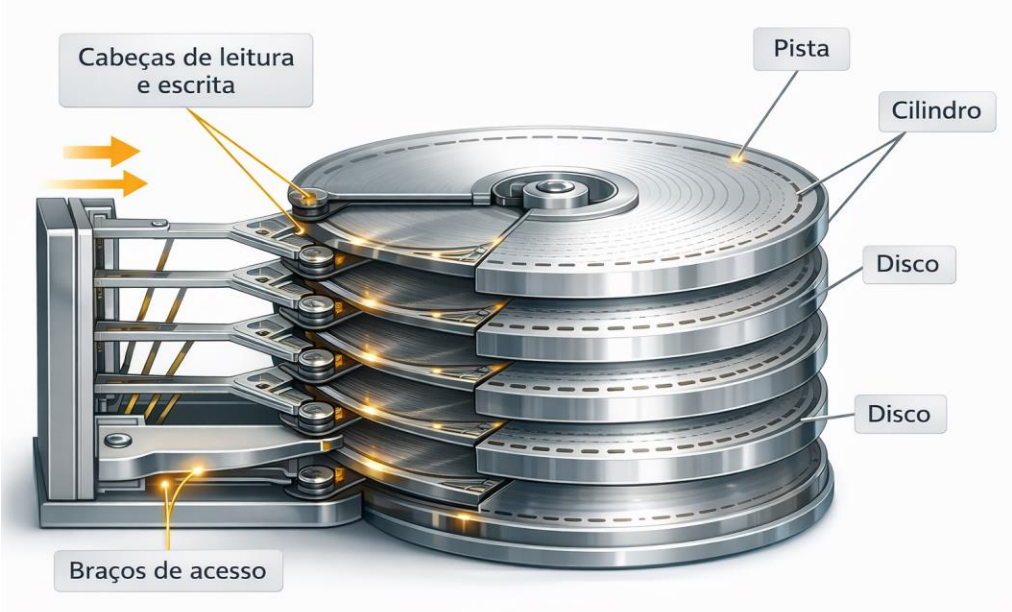
- O SO cria uma **máquina virtual mais simples**.

Hardware real	Abstração do SO
setores do disco	arquivos
CPU física	processos
controladores	dispositivos
memória física	memória virtual

Exemplo: Abstração de Arquivos

O disco físico possui:

- Trilhas
- Setores
- Cabeçotes



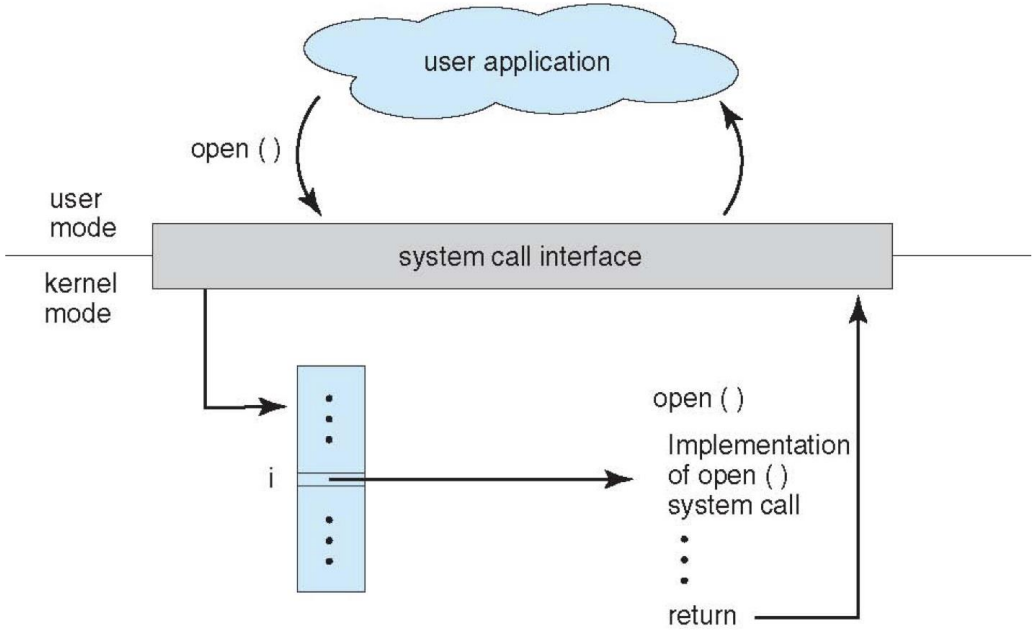
Exemplo: Abstração de Arquivos

Sistema operacional apresenta pra os usuários/programadores:

- arquivo.txt
- foto.jpg
- dados.csv

E disponibiliza os comandos:

- open()
- read()
- write()
- close()



Exemplo: Pseudocódigo em C

```
char Str[100];
FILE *arq;
arq = fopen("Arquivo.txt", "wt");  // Cria um arquivo texto para gravação

if (arq == NULL) // Se não conseguiu criar
{
   printf("Problemas na CRIACAO do arquivo\n");
   return;
}

strcpy(Str,"Linha de teste");
result = fputs(Str, arq);

if (result == EOF)
    printf("Erro na Gravacao\n");
fclose(arq);
```

```c
char Str[100];
FILE *arq;
arq = fopen("Arquivo.txt", "wt");  // Cria um arquivo texto para
gravação

if (arq == NULL) // Se não conseguiu criar
{
   printf("Problemas na CRIACAO do arquivo\n");
   return;
}

strcpy(Str,"Linha de teste");
result = fputs(Str, arq);

if (result == EOF)
    printf("Erro na Gravacao\n");
fclose(arq);
```

Exemplo: Código em C

```c
#include <stdio.h>
#include <string.h>
int main() {
    char Str[100];
    FILE *arq;
    int result;
    arq = fopen("Arquivo.txt", "wt"); // cria arquivo texto
    if (arq == NULL) {
        printf("Problemas na CRIACAO do arquivo\n");
        return 1;
    }
    strcpy(Str, "Linha de teste");
    result = fputs(Str, arq);
    if (result == EOF)
        printf("Erro na Gravacao\n");
    fclose(arq);
    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>
int main() {
    char Str[100];
    FILE *arq;
    int result;
    arq = fopen("Arquivo.txt", "wt"); // cria arquivo
texto
    if (arq == NULL) {
        printf("Problemas na CRIACAO do arquivo\n");
        return 1;
    }
    strcpy(Str, "Linha de teste");
```

```
    arq = fopen("Arquivo.txt", "wt"); // cria arquivo
texto
    if (arq == NULL) {
        printf("Problemas na CRIACAO do arquivo\n");
        return 1;
    }
    strcpy(Str, "Linha de teste");
    result = fputs(Str, arq);
    if (result == EOF)
        printf("Erro na Gravacao\n");
    fclose(arq);
    return 0;
}
```

Exemplo: Abstração de Processo

Programas em execução são representados como processos.

Um processo inclui:

- Código do programa
- Memória
- Estado da CPU
- Identificador (PID)

Exemplo:

- Chrome
- Firefox
- Spotify
- Word
- Writer

Exemplo: Abstração de Dispositivos

O hardware possui muitos dispositivos:

- Teclado
- Mouse
- Disco
- Rede
- Impressora

Exemplo em Linux:
/dev/sda
/dev/tty
/dev/null

Tudo pode ser tratado como **arquivo**

Chamadas de Sistema (System Call)

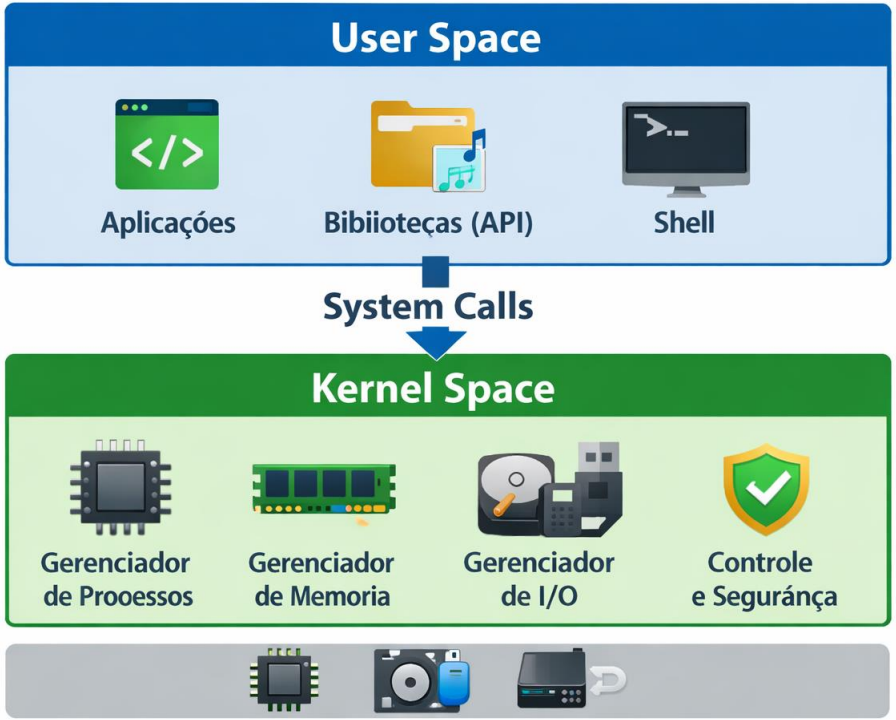
Aplicações acessam o SO através de **system calls**.

Exemplo em C:

```
int fd = open("dados.txt", O_RDONLY);
read(fd, buffer, 100);
close(fd);
```

Essas funções pedem ao **kernel** que execute operações.

User Space x Kernel Space



Vantagens da Máquina Estendida

Benefícios do modelo:

- ✓ Simplificação da programação
- ✓ Portabilidade de software
- ✓ Segurança do sistema
- ✓ Isolamento entre programas
- ✓ Padronização de hardware

Programadores focam na lógica da aplicação, não no hardware.

Sistema Operacional como Gerenciador de Recursos

O Sistema Operacional gerencia e distribui os recursos do computador entre múltiplos programas.

Recursos principais:

- CPU
- Memória
- Dispositivos de I/O
- Armazenamento

O Problema dos Recursos Limitados

Um computador possui recursos limitados:

- Apenas algumas CPUs
- Memória RAM finita
- Discos compartilhados
- Dispositivos de entrada e saída

Mas vários programas querem usar esses recursos ao mesmo tempo.

Papel do Sistema Operacional

O SO atua como administrador do computador.
Funções principais:

- Decidir quem usa o quê
- Decidir quando usar
- Evitar conflitos entre programas

Sem essa gestão:

- Programas poderiam travar o sistema
- Dados poderiam ser corrompidos
- Dispositivos poderiam ser usados simultaneamente

Exemplo: Gerenciamento do CPU

A CPU é um recurso crítico.

Problema:

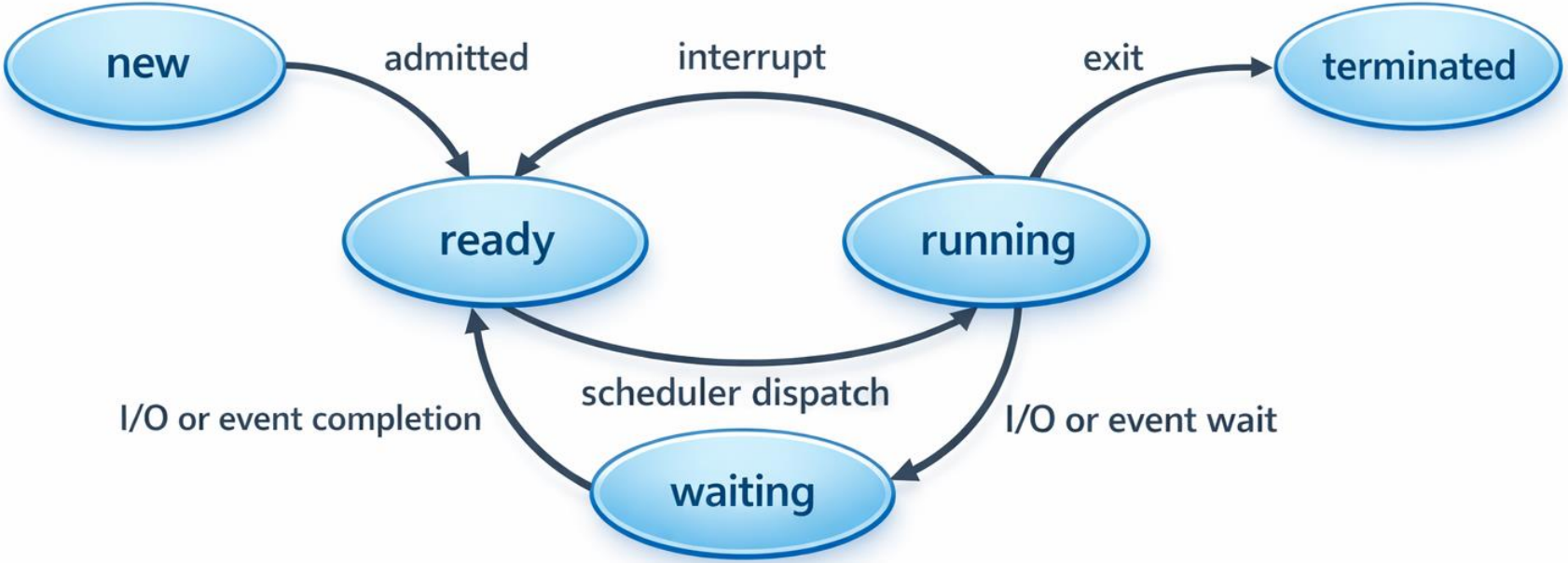
- Vários processos querem executar simultaneamente.

Solução:

- Escalonamento de processos

O SO alterna entre eles usando **time sharing**.

Exemplo: Gerenciamento do CPU



Escalonamento de Processos

O sistema operacional usa algoritmos para decidir:

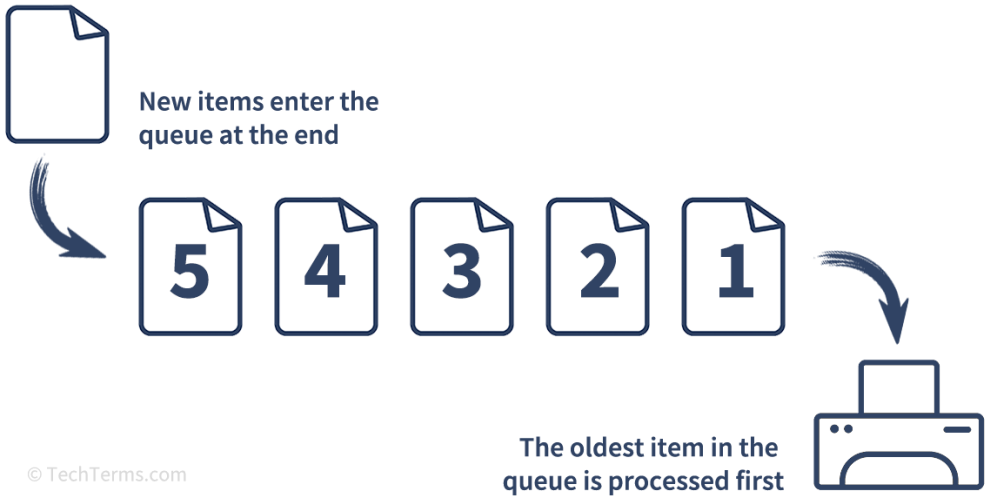
- Qual processo executa
- Por quanto tempo

Exemplos de algoritmos:

- FIFO
- Round Robin
- Shortest Job First

Objetivos:

- Justiça
- Eficiência
- tempo de resposta



Gerenciamento de Memória

A memória RAM também é limitada.

O SO precisa:

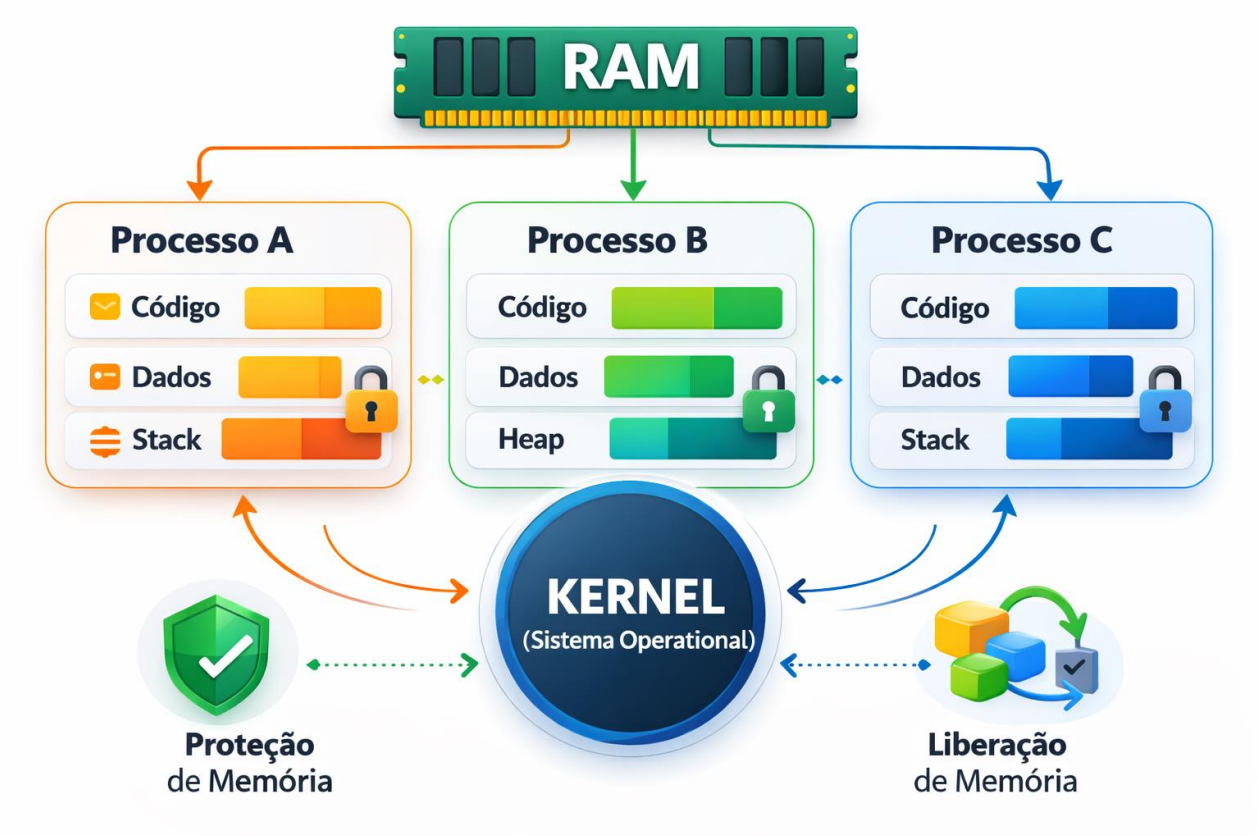
- Alocar memória para processos
- Proteger memória entre processos
- Liberar memória quando não for usada

Exemplo:

- Processo A → 200 MB
- Processo B → 150 MB
- Processo C → 300 MB

Cada processo possui **espaço isolado.**

Gerenciamento de Memória



Memória Virtual

O SO cria a abstração de memória virtual.

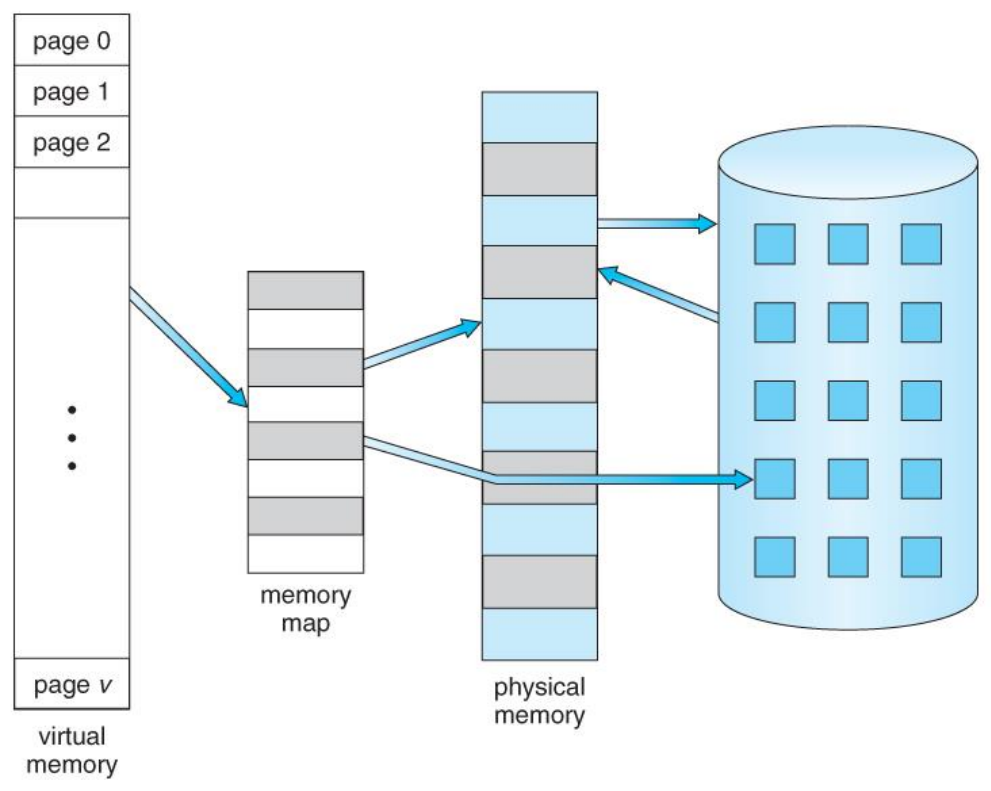
Benefícios:

- Programas podem usar mais memória do que a RAM disponível
- Isolamento entre processos
- Maior segurança

Mecanismo usado:

- Paginação
- swap

Exemplo: Memória Virtual



Gerenciamento de Dispositivos de I/O

Os dispositivos de entrada e saída são compartilhados.

Exemplos:

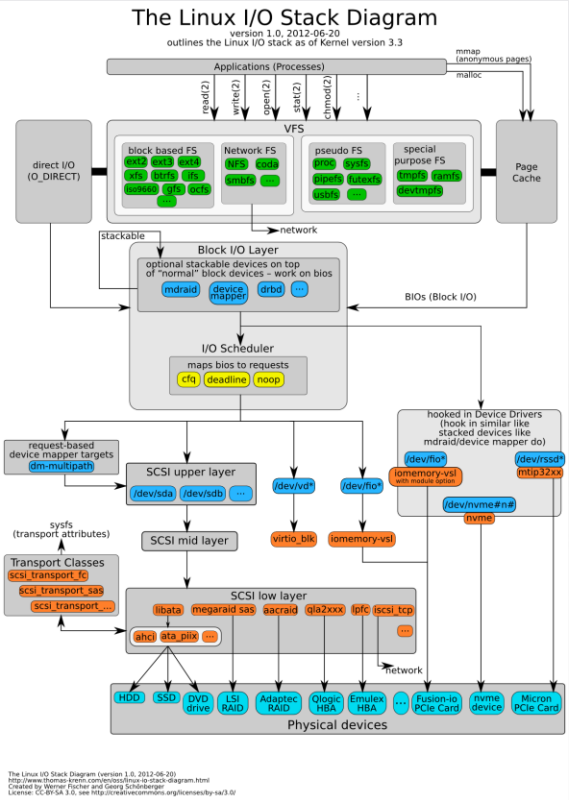
- Discos
- Impressoras
- Rede
- Teclado

O SO utiliza:

- ❑Drivers
- ❑filas de requisição
- ❑Buffers

Para organizar o acesso aos dispositivos.

Exemplo: Gerenciamento de I/O



Proteção e Isolamento

O gerenciamento de recursos também envolve segurança.

O SO precisa garantir que:

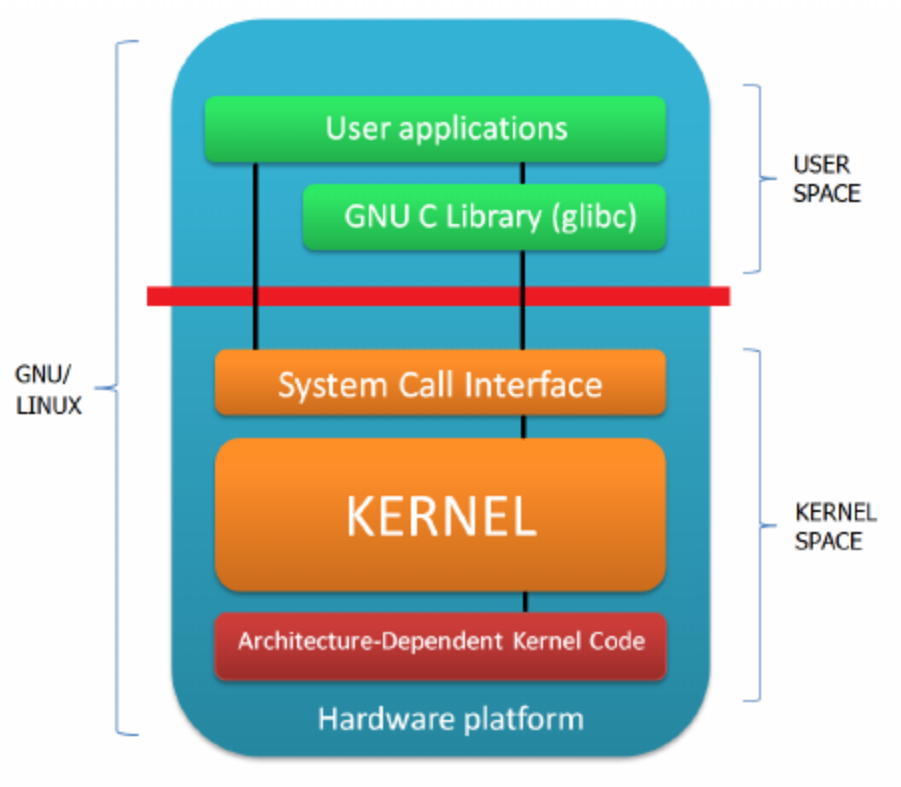
- Um processo não acesse memória de outro
- Programas não controlem hardware diretamente
- Falhas de um processo não derrubem o sistema

Isso é feito com:

- **modo usuário**
- **modo kernel**

Proteção e Isolamento

- O SO é dividido em diversas partes
- O kernel (núcleo) é a parte mais baixo nível do sistema operacional
- Gerencia recursos do sistema, hardware etc.
- Outra possível parte de um SO é o conjunto de programas utilitários disponibilizados, como compiladores e editor de texto



Mão na massa! System Call na prática.

Ferramenta que iremos usar:

- https://www.onlinegdb.com/

Linguagem de programação: C

Objetivo:

- **Entender** o SO como máquina estendida

Nosso primeiro programa C com System Call

Objetivo:

Criar um programa em linguagem C que leia um arquivo de texto e apresente o resultado na tela com:

- O conteúdo de cada linha.
- O número de linhas.
- A quantidade de caracteres.

Código Base

Abrindo e fechando um arquivo:

```
#include <stdio.h>

int main() {

    FILE *f = fopen("dados.txt", "r");
    if (f == NULL) {
        printf("Erro ao abrir arquivo\n");
        return 1;
    }
    fclose(f);
}
```

Código Base

```c
#include <stdio.h>

int main() {

    FILE *f = fopen("dados.txt", "r");

    if (f == NULL) {
        printf("Erro ao abrir arquivo\n");
        return 1;
    }

    fclose(f);
}
```

Código Base

Criando um arquivo de textp com conteúdo “hello”:

```
#include <stdio.h>


int main() {
    FILE *f = fopen("dados.txt", "w");
    if (f == NULL) {

        printf("Erro ao abrir arquivo\n");

        return 1;

    }
    fprintf(f, "Hello");
    fclose(f);
    return 0;

}
```

Hora de perguntar, dúvidas

Próxima aula

Conceitos Fundamentais de SO

- Estruturas de Sistemas Operacionais:
  - Organização do kernel e dos componentes principais. Arquiteturas de Kernel: monolítico, microkernel e arquitetura em camadas em detalhes.
- Kernels modernos e híbridos: exemplos de Linux, Windows e Android.
- Atividade prática: análise comparativa de arquiteturas de kernel reais.